TRABAJO PRÁCTICO INTEGRADOR

NumPy — Red de monitoreo de calidad del aire

ASIGNATURA	MODALIDAD
Programación II · 3.er año	**Individual o en parejas**
ENTREGA	EXTENSIÓN DEL INFORME
tp_numpy_apellido.py + informe	**Máximo 2 páginas**

RESTRICCIÓN CENTRAL Salvo en la Parte 1 (donde se pide comparar), **no se permiten bucles `for` / `while` para procesar datos**. Todo el análisis debe resolverse con operaciones vectorizadas de NumPy.

1. Contexto

Una ciudad instaló **6 sensores** de material particulado (PM2.5, en µg/m^3) distribuidos en distintos barrios. Cada sensor registra **una medición por hora durante 14 días** (14 × 24 = 336 mediciones por sensor).

Sos el/la analista de datos contratado/a para producir un reporte que le permita al municipio decidir en qué barrios y en qué momentos declarar alerta ambiental.

Los datos se generan de forma simulada (y reproducible) con la plantilla **`tp_plantilla.py`**. Cada alumno **debe cambiar la semilla** a su número de legajo, de modo que los resultados difieran entre estudiantes.

Dato	Descripción
`barrios`	`np.array` de 6 nombres (`str`)
`mediciones`	matriz `float` de forma `(6, 336)`: filas = sensores, columnas = horas
`umbral`	35 µg/m^3 (límite diario recomendado)

> **NOTA** Algunas mediciones pueden ser `-1`: es el código que usa el sensor cuando **falla** y no registra el dato. Tenerlo en cuenta.

2. Consigna

Parte 1 — Exploración y vectorización (≈ 15 %)

Conceptos: *arrays, propiedades, dtype, vectorización, listas vs. arrays*

1. Mostrar `shape`, `ndim`, `size`, `dtype` e `itemsize` de `mediciones`, y el total de bytes que ocupa (`nbytes`).
2. Convertir una copia de `mediciones` a `float32` con `astype()` y calcular cuánta memoria se ahorra. Explicar en el informe qué se pierde (si algo) al hacerlo.
3. Calcular el **promedio global** de dos maneras: (a) con un bucle `for` sobre una lista de Python, (b) con `np.mean`. Medir ambos tiempos con `time.perf_counter()` y comentar la diferencia. Repetir la medición con un array de 1.000.000 de valores (`rng.random(1_000_000)`) y explicar por qué con pocos datos la ventaja de NumPy puede no notarse.

Parte 2 — Limpieza de datos: copias, vistas y filtrado booleano (≈ 20 %)

Conceptos: *copy vs. view, máscaras booleanas, np.where, condiciones múltiples, np.nan*

4. Crear `datos = mediciones.copy()`. Demostrar con un ejemplo (comentado) qué hubiera pasado si en lugar de `copy()` se hubiera usado `view()` o una simple asignación `datos = mediciones`.
5. Construir una máscara `fallas = datos == -1`. Informar cuántas fallas hubo en total, **por sensor** y en qué posiciones (`np.where`) ocurrió la primera falla del sensor con más fallas.

6. Reemplazar los `-1` por `np.nan`. A partir de ahí, usar las funciones que ignoran NaN (`np.nanmean`, `np.nanmax`, `np.nanstd`, etc.).
7. Detectar **valores atípicos** (outliers): mediciones que superen en más de 3 desvíos estándar a la media de su sensor. Pista: `broadcasting` entre una matriz `(6, 336)` y vectores de forma `(6, 1)`.

Parte 3 — Reorganización: shape, reshape, indexación y slicing (≈ 20 %)

Conceptos: *reshape, -1, índices, slicing en matrices, recorrido*

8. Reorganizar `datos` en un array 3D de forma `(6, 14, 24)` → (sensor, día, hora). Verificar con `shape` y comprobar que `datos3d[2, 5, 10]` coincide con la medición correcta del array original.
9. Usando **slicing** (sin bucles), obtener:
   - Las mediciones del sensor 0 durante la **primera semana**.
   - Las mediciones de **todas las estaciones** entre las 7 y las 9 h (hora pico matutina) de todos los días.
   - Los **últimos 3 días** de los sensores con índice par.
10. Recorrer con `np.nditer` los primeros 5 valores de la primera fila para imprimirlos y explicar en un comentario en qué se diferencia de un `for` común sobre una matriz.

Parte 3b — Unión y división (≈ 5 %)

Conceptos: *concatenate, array_split, sort*

11. Dividir `datos3d` en dos bloques con `np.array_split` (semana 1 y semana 2) y volver a unirlos con `np.concatenate` en el orden inverso. Verificar con `np.array_equal` que el resultado es el esperado.
12. Ordenar los barrios de **mayor a menor** contaminación promedio (`np.argsort` sobre los promedios) e imprimir el ranking.

Parte 4 — Estadística por ejes (≈ 25 %)

Conceptos: *axis, mean, median, std, min, max, sum, argmax*

13. Construir una tabla (matriz) `(6, 14)` con el **promedio diario** de cada sensor (pista: `axis`).
14. Calcular y mostrar, para cada barrio: promedio, mediana, desvío estándar, mínimo y máximo (ignorando NaN). En el informe, interpretar **dónde conviene mirar la mediana en lugar del promedio** y qué dice el desvío estándar sobre la estabilidad de cada barrio.
15. Responder con código (no a ojo):
    - ¿Qué barrio tuvo el **día más contaminado** y cuál fue ese día? (`np.argmax`, `np.unravel_index`)
    - ¿Cuál es la **hora del día** con mayor contaminación promedio considerando todos los sensores y días?
    - ¿Cuántos días **superaron el umbral** de 35 µg/m^3 en cada barrio?
16. Calcular el **porcentaje de horas en alerta** (> `umbral`) por barrio y mostrarlo ordenado.

Parte 5 — Simulación con números aleatorios (≈ 10 %)

Conceptos: *default_rng, integers, normal/random, choice*

17. Con `np.random.default_rng(legajo)`, simular que el municipio instala **un séptimo sensor**: generar sus 336 mediciones con una distribución normal (`rng.normal`) cuya media y desvío sean los **de toda la red**. Agregarlo a la matriz con `np.vstack`/`concatenate` y recalcular el promedio de la red.
18. Elegir al azar con `rng.choice` **10 mediciones** de un sensor al azar para una "auditoría" y mostrar sus posiciones y valores.

Parte 6 — Reporte final (≈ 5 % código + informe)

19. Implementar la función:

```
def clasificar_aire(promedios_diarios: np.ndarray) -> np.ndarray:
    """Devuelve un array de str con 'Bueno', 'Moderado', 'Malo' o 'Crítico'
    según el promedio diario (< 12, < 35, < 55, >= 55 µg/m³).
    Debe estar vectorizada (pista: np.select o np.where anidados)."""
```

20. Aplicarla a la tabla `(6, 14)` del punto 13 y mostrar, por barrio, **cuántos días cayeron en cada categoría**.

21. Imprimir un **reporte final** con formato (título con la función `titulo()`, tablas alineadas) que incluya: el barrio recomendado para intervención prioritaria, el día más crítico y la franja horaria de mayor riesgo.

3. Informe (máx. 2 páginas)

Debe incluir:

- Las respuestas interpretativas de los puntos **2, 14** y la comparación de tiempos del punto **3**.
- Una captura (o texto) del reporte final.
- Un párrafo de **reflexión**: ¿qué limitaciones tienen estos datos simulados? ¿Qué decisión **no** tomarías solo con este análisis?

4. Requisitos de código

- Una función `main()` y bloque `if __name__ == "__main__":`.
- Cada parte del TP en su propia función (`parte_1()`, `parte_2()`, ...) con **docstring** que explique qué hace.
- Nombres de variables descriptivos en español o inglés (no mezclar).
- Comentarios solo donde aporten (no repetir el código).
- Entregar el código ejecutable de principio a fin sin errores.

5. Criterios de evaluación

Criterio	Peso
Corrección de resultados	35 %
Uso adecuado de NumPy (vectorización, `axis`, máscaras, sin bucles innecesarios)	25 %
Claridad y documentación del código	15 %
Interpretación en el informe	15 %
Prolijidad del reporte final	10 %

Bonus (+10 %): graficar con `matplotlib` el promedio diario por barrio, marcando el umbral con una línea horizontal.

6. Mapa de conceptos cubiertos

Concepto de la clase	Puntos del TP
Arrays y propiedades (`shape`, `ndim`, `dtype`, `size`)	1, 8
Listas vs. arrays / vectorización	3, 19
Tipos y `astype`	2
`copy` vs. `view`	4
Indexación y slicing	8, 9
`reshape`	8
Recorrido (`nditer`)	10
Unión y división	11
Búsqueda (`where`) y ordenamiento	5, 12, 16
Filtrado booleano y condiciones múltiples	5, 7, 16

Concepto de la clase	Puntos del TP
Estadística y `axis`	13–16
Números aleatorios	17, 18
Ejemplo integrador	19–21

7. Pautas de integridad académica

Se permite consultar el tutorial, la documentación oficial de NumPy y los ejemplos de clase. Todo código que no sea propio debe estar citado. Cada integrante deberá poder **explicar y modificar en vivo** cualquier parte de su entrega durante una breve defensa oral.

8. Archivo de apoyo: plantilla Python

La consigna se acompaña del archivo **`tp_plantilla.py`**, que contiene `generar_datos()`, `titulo()`, `main()` y las funciones que deben completar los estudiantes. La plantilla se entrega **por separado y sin soluciones** para que pueda abrirse y ejecutarse como código fuente.

Personalización obligatoria: reemplazar el valor de la constante `LEGAJO = 12345` por el número de legajo correspondiente. No modificar `generar_datos()`.